

Student Guide: Building Your Quiz Repository Step by Step

Automated Software Testing (192-211)

This guide shows you how to set up ONE GitHub repository for all your quizzes and build it up progressively as you work. You do not need GitHub Classroom or any paid tool -- just a normal free GitHub account.

Why work this way? Committing your work as you go (not all at once at the end) does two things for you: it saves your progress so you never lose work, and it shows your real problem-solving process. It also makes grading fair and fast.

Part A: One-Time Setup (do this once for the whole course)

A.1 Create your GitHub account

If you do not have one, sign up free at github.com.

A.2 Create your course repository

1. On github.com, click the + (top right) -> **New repository**.
2. Name it exactly using this format:

`st211-quizzes-YOUR_STUDENT_ID`

Example: `st211-quizzes-6505001`

3. Set it to **Private**.
4. Tick **"Add a README file"**.
5. Click **Create repository**.

A.3 Add your instructor

So your instructor can see and grade your work:

1. In your repository, go to **Settings** -> **Collaborators**.
2. Click **Add people** and enter the instructor's GitHub username.

A.4 Get the repository onto your computer

Copy your repository's URL (green **Code** button -> HTTPS), then in your terminal:

```
git clone https://github.com/YOUR_USERNAME/st211-quizzes-YOUR_STUDENT_ID.git
cd st211-quizzes-YOUR_STUDENT_ID
```

You now have a folder for all your quizzes. You will work inside it all term.

Part B: The Folder Layout (keep it tidy)

Every quiz gets its own folder. Inside each quiz folder you put your Python solution, your tests, and a short README. Your repository should look like this:

```
st211-quizzes-6505001/  
■■■ README.md # your name, student ID, and a list of quizzes  
■■■ quiz-01/  
■ ■■■ solution.py # your Python solution  
■ ■■■ test_solution.py # your pytest tests  
■ ■■■ README.md # how to run it + a sample of the output  
■■■ quiz-02/  
■ ■■■ solution.py  
■ ■■■ test_solution.py  
■ ■■■ README.md  
■■■ ...
```

Keeping this exact layout matters: it is part of your grade, and it means your instructor can find and run your work the same way for everyone.

Part C: Building a Quiz Progressively (the important part)

Do NOT write the whole quiz and push it once at the end. Instead, **commit each time you make a meaningful step forward.** Here is the natural rhythm for one quiz.

Step 1: Start the quiz folder

```
mkdir quiz-01  
cd quiz-01
```

Create your starting files (empty is fine to begin):

```
touch solution.py test_solution.py README.md
```

Commit this starting point:

```
git add .  
git commit -m "quiz-01: set up folder and empty files"
```

Step 2: Write a first attempt at the solution

Write your first version of `solution.py` -- even if it is not finished. Then:

```
git add solution.py  
git commit -m "quiz-01: first working version of the main function"
```

Step 3: Add a test, watch it guide you

Write one test in `test_solution.py`, run it, and see if it passes:

```
python3 -m pytest test_solution.py
```

Commit your test:

```
git add test_solution.py  
git commit -m "quiz-01: add tests for the basic cases"
```

Step 4: Fix and improve

As you fix bugs and handle more cases, commit each real improvement:

```
git add .
git commit -m "quiz-01: handle empty input edge case"
```

```
git add .
git commit -m "quiz-01: fix off-by-one in the loop"
```

Step 5: Write the quiz README

In `quiz-01/README.md`, write two or three lines: what the quiz asked, how to run your solution, and a sample of the output. Commit it:

```
git add README.md
git commit -m "quiz-01: add run instructions and sample output"
```

Step 6: Push to GitHub

Whenever you want to save your work online (do this often, not just at the end):

```
git push
```

That is the whole cycle. Small commits as you go, push often. By the time the quiz is due, your work is already saved and your history tells the story of how you built it.

Part D: What "Good" Looks Like

A healthy quiz has a history like this -- several small commits showing progress:

```
quiz-01: set up folder and empty files
quiz-01: first working version of the main function
quiz-01: add tests for the basic cases
quiz-01: handle empty input edge case
quiz-01: fix off-by-one in the loop
quiz-01: add run instructions and sample output
```

Compare that to a single commit called "upload" added five minutes before the deadline. The first shows real work. The second does not -- and it will be looked at more closely.

Good commit messages say what changed:

- * Good: `quiz-01: handle negative numbers`
 - * Not helpful: `update, stuff, asdf, final final v2`
-

Part E: Deadlines (how they work here)

There are no special commands to learn for deadlines. Your grade uses the time of your **last commit before the deadline**. GitHub records the exact time of every commit automatically.

- * Anything you commit and push before the deadline counts.
- * If your latest work is committed after the deadline, the late penalty applies to that quiz.

* **Push before the deadline.** A commit sitting only on your laptop, not pushed to GitHub, cannot be seen or graded.

How You Are Graded

Each quiz is out of 100 points:

Component	Weight	What it means
Correctness (tests pass)	60%	Your solution produces the right results and passes the tests.
Code quality	25%	Readable code, sensible structure, edge cases handled.
Process (git and layout)	15%	Correct folder layout, more than one commit showing real progress, and a quiz README.

The biggest share is simply having working, tested code. The process marks reward the good habits this guide teaches: build up your work in small commits and keep your folders tidy.

Part F: Quick Reference

Everyday commands you will use:

```
git status # what have I changed?
git add . # stage all my changes
git commit -m "quiz-01: ..." # save a snapshot with a message
git push # send it to GitHub
git log --oneline # see my commit history
```

The rule of thumb: ****every time something works or you fix something, commit it.**
Push at the end of every work session.**